

New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

06-gold

initial-feature

Python

File Edit View Run Help [Last edit was now](#) [Provide feedback](#)

Run all

Connect

Schedule

Share

Cmd 1

1 %run ./01-config

Cmd 2

```
1 class Upsserter:
2     def __init__(self, merge_query, temp_view_name):
3         self.merge_query = merge_query
4         self.temp_view_name = temp_view_name
5
6     def upsert(self, df_micro_batch, batch_id):
7         df_micro_batch.createOrReplaceTempView(self.temp_view_name)
8         df_micro_batch.jdf.sparkSession().sql(self.merge_query)
```

Cmd 3

1

Python

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

06-gold

initial-feature

Python



File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all

Connect

Schedule

Share

```
1 class Gold():
2     def __init__(self, env):
3         self.Conf = Config()
4         self.test_data_dir = self.Conf.base_dir_data + "/test_data"
5         self.checkpoint_base = self.Conf.base_dir_checkpoint + "/checkpoints"
6         self.catalog = env
7         self.db_name = self.Conf.db_name
8         self.maxFilesPerTrigger = self.Conf.maxFilesPerTrigger
9         spark.sql(f"USE {self.catalog}.{self.db_name}")
10
11     def upsert_workout_bpm_summary(self, once=True, processing_time="15 seconds", startingVersion=0):
12         from pyspark.sql import functions as F
13
14         #Idempotent - Once a workout session is complete, It doesn't change. So insert only the new records
15         query = f"""
16         MERGE INTO {self.catalog}.{self.db_name}.workout_bpm_summary a
17         USING workout_bpm_summary_delta b
18         ON a.user_id=b.user_id AND a.workout_id = b.workout_id AND a.session_id=b.session_id
19         WHEN NOT MATCHED THEN INSERT *
20         """
21
22         data_upserter=Upsserter(query, "workout_bpm_summary_delta")
23
24         df_users = spark.read.table(f"{self.catalog}.{self.db_name}.user_bins")
```

New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

06-gold

initial-feature

Python

☆

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all

Connect

Schedule

Share

```
11 def upsert_workout_bpm_summary(self, once=True, processing_time="15 seconds", startingVersion=0):
12     from pyspark.sql import functions as F
13
14     #Idempotent - Once a workout session is complete, It doesn't change. So insert only the new records
15     query = f"""
16     MERGE INTO {self.catalog}.{self.db_name}.workout_bpm_summary a
17     USING workout_bpm_summary_delta b
18     ON a.user_id=b.user_id AND a.workout_id = b.workout_id AND a.session_id=b.session_id
19     WHEN NOT MATCHED THEN INSERT *
20     """
21
22     data_upserter=Upsserter(query, "workout_bpm_summary_delta")
23
24     df_users = spark.read.table(f"{self.catalog}.{self.db_name}.user_bins")
25
26     df_delta = (spark.readStream
27                 .option("startingVersion", startingVersion)
28                 .option("ignoreDeletes", True)
29                 .option("withEventTimeOrder", "true")
30                 .option("maxFilesPerTrigger", self.maxFilesPerTrigger)
31                 .table(f"{self.catalog}.{self.db_name}.workout_bpm")
32                 .withWatermark("end_time", "30 seconds")
33                 .groupBy("user_id", "workout_id", "session_id", "end_time")
34                 .agg(F.min("heartrate").alias("min_bpm"), F.mean("heartrate").alias("avg_bpm"),
35                     F.max("heartrate").alias("max_bpm"), F.count("heartrate").alias
36                     ("num_recordings"))
```


New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

06-gold

initial-feature

Python



File Edit View Run Help Last edit was 11 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
data_upserter=Upsserter(query, "workout_bpm_summary_delta")
```

```
df_users = spark.read.table(f"{self.catalog}.{self.db_name}.user_bins")
```

```
df_delta = (spark.readStream
```

```
    .option("startingVersion", startingVersion)
```

```
    #.option("ignoreDeletes", True)
```

```
    #.option("withEventTimeOrder", "true")
```

```
    #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
```

```
    .table(f"{self.catalog}.{self.db_name}.workout_bpm")
```

```
    .withWatermark("end_time", "30 seconds")
```

```
    .groupBy("user_id", "workout_id", "session_id", "end_time")
```

```
    .agg(F.min("heartrate").alias("min_bpm"), F.mean("heartrate").alias("avg_bpm"),
```

```
         F.max("heartrate").alias("max_bpm"), F.count("heartrate").alias
```

```
         ("num_recordings"))
```

```
    .join(df_users, ["user_id"])
```

```
    .select("workout_id", "session_id", "user_id", "age", "gender", "city", "state", "min_bpm",
```

```
           "avg_bpm", "max_bpm", "num_recordings")
```

```
)
```

```
stream_writer = (df_delta.writeStream
```

```
    .foreachBatch(data_upserter.upsert)
```

```
    .outputMode("append")
```

```
    .option("checkpointLocation", f"{self.checkpoint_base}/workout_bpm_summary")
```

```
    .queryName("workout_bpm_summary_upsert_stream")
```

```
)
```

New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

06-gold

initial-feature

Python

☆

File Edit View Run Help Last edit was 13 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
48
49     if once == True:
50         return stream_writer.trigger(availableNow=True).start()
51     else:
52         return stream_writer.trigger(processingTime=processing_time).start()
53
54
55     def upsert(self, once=True, processing_time="5 seconds"):
56         import time
57         start = int(time.time())
58         print(f"\nExecuting gold layer upsert ...")
59         self.upsert_workout_bpm_summary(once, processing_time)
60         if once:
61             for stream in spark.streams.active:
62                 stream.awaitTermination()
63         print(f"Completed gold layer upsert {int(time.time()) - start} seconds")
64
65
66     def assert_count(self, table_name, expected_count, filter="true"):
67         print(f"Validating record counts in {table_name}...", end='')
68         actual_count = spark.read.table(f"{self.catalog}.{self.db_name}.{table_name}").where(filter).count()
69         assert actual_count == expected_count, f"Expected {expected_count:,} records, found {actual_count:,} in {table_name} where {filter}"
70         print(f"Found {actual_count:,} / Expected {expected_count:,} records where {filter}: Success")
71
72     def assert_rows(self, location, table_name, sets):
73         print(f"Validating records in {table_name}...", end='')
```


New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

06-gold

initial-feature

Python

☆

File Edit View Run Help Last edit was 13 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
54
55 def upsert(self, once=True, processing_time="5 seconds"):
56     import time
57     start = int(time.time())
58     print(f"\nExecuting gold layer upsert ...")
59     self.upsert_workout_bpm_summary(once, processing_time)
60     if once:
61         for stream in spark.streams.active:
62             stream.awaitTermination()
63     print(f"Completed gold layer upsert {int(time.time()) - start} seconds")
64
65
66 def assert_count(self, table_name, expected_count, filter="true"):
67     print(f"Validating record counts in {table_name}...", end='')
68     actual_count = spark.read.table(f"{self.catalog}.{self.db_name}.{table_name}").where(filter).count()
69     assert actual_count == expected_count, f"Expected {expected_count:,} records, found {actual_count:,} in {table_name} where {filter}"
70     print(f"Found {actual_count:,} / Expected {expected_count:,} records where {filter}: Success")
71
72 def assert_rows(self, location, table_name, sets):
73     print(f"Validating records in {table_name}...", end='')
74     expected_rows = spark.read.format("parquet").load(f"{self.test_data_dir}/{location}_{sets}.parquet").collect()
75     actual_rows = spark.table(table_name).collect()
76     assert expected_rows == actual_rows, f"Expected data mismatches with the actual data in {table_name}"
77     print(f"Expected data matches with the actual data in {table_name}: Success")
78
79
```

+ New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

06-gold initial-feature Python ☆

File Edit View Run Help Last edit was 13 minutes ago Provide feedback

▶ Run all

● Connect ▼

📅 Schedule

Share

```
73     print(f"Validating records in {table_name}...", end='')
74     expected_rows = spark.read.format("parquet").load(f"{self.test_data_dir}/{location}_{sets}.parquet").collect()
75     actual_rows = spark.table(table_name).collect()
76     assert expected_rows == actual_rows, f"Expected data mismatches with the actual data in {table_name}"
77     print(f"Expected data matches with the actual data in {table_name}: Success")
78
79
80     def validate(self, sets):
81         import time
82         start = int(time.time())
83         print(f"\nValidating gold layer records..." )
84         self.assert_rows("7-gym_summary", "gym_summary", sets)
85         if sets>1:
86             self.assert_rows("8-workout_bpm_summary", "workout_bpm_summary", sets)
87         print(f"Gold layer validation completed in {int(time.time()) - start} seconds")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text